

Trabajos Prácticos: Placa BluePill

Los siguientes trabajos prácticos se desarrollarán sobre una placa BluePill. Algunos ejercicios requerirán elementos adicionales; se recomienda utilizar un protoboard para alojar la placa y garantizar el libre acceso a su pinout.

Ejercicio Práctico N.º 1: Interrupciones Externas y el NVIC 2/06/26 ✓

El uso de un botón para desencadenar una acción en el sistema es un ejemplo clásico de evento asíncrono. La alternativa más simple consiste en verificar periódicamente si el botón fue presionado (*polling*); sin embargo, en este ejercicio se configura el *Nested Vectored Interrupt Controller* (NVIC) para gestionar dicho evento mediante interrupción.

Mediante el botón se modificará el estado de un LED (encendido/apagado) conectado a uno de los pines GPIO disponibles. El LED parpadeante del TP1 se mantiene activo durante todo el ejercicio.

El uso de interrupciones en sistemas embebidos es fundamental para comprender cómo se logra baja latencia de respuesta.

- **Objetivo:** Registrar una función en la tabla de vectores y gestionar los registros de prioridad del NVIC.
- **Periféricos:** GPIO (entrada), AFIO (*Alternate Function I/O*) y NVIC.
- **Contenido:**
 - Configuración del NVIC directamente mediante los registros ISER (*Interrupt Set-Enable Registers*).
 - Manejo de la prioridad de interrupción.
 - Implementación de la rutina de servicio (ISR) en la tabla de vectores.

Ejercicio Práctico N.º 2: Conversión Analógica-Digital (ADC) 2/06/26

El ADC es uno de los periféricos más utilizados en sistemas embebidos. En este ejercicio se implementará la lectura del sensor de temperatura interna de la BluePill; se emplearán 5 LEDs conectados a GPIO para indicar rangos de temperatura. Mediante el botón del ejercicio anterior se alternará entre la lectura del sensor interno y la de un ADC con canal externo conectado a un potenciómetro.

El LED parpadeante del TP1 se mantiene activo durante todo el ejercicio.

- **Objetivo:** Comprender el proceso de conversión ADC y la configuración de sus registros en el microcontrolador.
- **Periféricos:** ADC1, GPIO (modo analógico).
- **Contenido:**
 - Configuración de ADC_CR2 (ADON, SWSTART).
 - Selección de canales en ADC_SQR3.
 - Lectura por *polling* del flag EOC (*End of Conversion*).
 - Implementación de la lectura del sensor de temperatura interno del chip.

Ejercicio Práctico N.º 3: Migración a CMSIS y Gestión de Tiempos

En este ejercicio se introduce el estándar CMSIS (*Cortex Microcontroller Software Interface Standard*), que permite abandonar el uso de punteros manuales y trabajar con las estructuras de datos definidas por el fabricante.

Además, se reemplaza el retardo por bucle del LED parpadeante: se configura el SysTick —un timer interno del núcleo Cortex-M3— para generar una base de tiempo precisa de 1 ms, que se incorpora como retardo para el LED.

- **Objetivo:** Portar el TP2 utilizando las estructuras definidas en `stm32f10x.h`.
- **Periféricos:** SysTick, GPIO y ADC.
- **Contenido:**
 - Uso de la estructura `GPIO_TypeDef`.
 - Configuración del timer del sistema (SysTick) para generar retardos precisos de 1 ms.
 - Análisis comparativo de legibilidad y mantenimiento: Bare Metal puro vs. CMSIS.

Ejercicio Práctico N.º 4: Comunicación Serie (UART) y Protocolos

La UART es uno de los periféricos más comunes y versátiles en sistemas embebidos, ya que permite enviar y recibir caracteres de forma sencilla para transmitir datos de estado o recibir comandos desde una terminal.

Se deberá configurar el periférico USART para comunicación bidireccional. El objetivo es desarrollar una pequeña interfaz de comandos por consola (UART a USB) que permita interactuar con la BluePill en tiempo real desde la PC.

- **Objetivo:** Establecer comunicación bidireccional entre la BluePill y la PC.
- **Periféricos:** USART, NVIC (interrupciones de recepción).
- **Contenido:**
 - Cálculo del *Baud Rate Register* (BRR).
 - Implementación de una función `print_string` básica.
 - Uso de interrupciones para recibir comandos sin bloquear el flujo principal del programa.

Ejercicio Práctico N.º 5: Control de Potencia con Timers (PWM)

En este ejercicio se utilizan los timers de propósito general para generar una señal PWM. Se deberán configurar los registros de autorrecarga y comparación, lo que permitirá controlar la potencia entregada a una carga —por ejemplo, el brillo de un LED o la velocidad de un motor mediante una llave H— de forma eficiente por hardware, sin consumir ciclos de CPU.

- **Objetivo:** Generar señales moduladas en ancho de pulso para controlar la intensidad de un LED.
- **Periféricos:** TIMx (*General Purpose Timers*).
- **Contenido:**
 - Configuración del Pre-scaler y el *Auto-reload Register* (ARR).
 - Configuración del modo PWM en el registro CCMRx.
 - Implementación de un efecto *fading* por software mediante la variación del *Duty Cycle*.